

Speech Control for Smean's TTS: Experiments, Findings, and Results

Prepared by **R&D Department**

8 September 2026

Contents

1 Executive Summary	—
2 Background	—
2.1 Problem	—
2.2 Model Selection: Why VoxCPM2	—
3 Speech Control Framework	—
3.1 Scope and Definitions	—
3.2 Prosody	—
3.3 Emotion	—
3.4 Non-Verbal Vocalization	—
4 Experiments	—
4.1 Baseline Prosodic Control on Khmer	—
4.2 Tag Conditioning: Capability Test	—
4.3 Tag Conditioning: Generalization Test	—
5 Findings	—
References	—

1 Executive Summary

This report covers speech control for Khmer text-to-speech on VoxCPM2, the model this project has adopted for Khmer (Section 2.2): the ability to specify how an utterance is spoken, not just what is said. It is a living document — it records what has been measured so far and will be updated as further experiments are run, rather than fixing a conclusion up front.

Three results stand out so far.

- (1) **Prosody (pitch, energy, speaking rate) already works on Khmer, out of the box.** A prompt describing a high pitch moves Khmer pitch by 106 Hz, about four times what this project's own labelled corpus could teach (Section 4.1).
- (2) **Emotion is blocked on data, not on the model.** Reliable emotional TTS work depends on acted, labelled emotional speech corpora, and no such corpus exists for Khmer (Section 3.3).
- (3) **Non-verbal vocalization's tag mechanism can be taught, but the first training recipe tried does not generalize.** A cheap overfitting test shows the model *can* learn to place a tag like [cough] at a specific point in a sentence (Section 4.2). A held-out test on the same recipe shows that capability does not transfer to sentences the model was not trained on (Section 4.3).

2 Background

2.1 Problem

Getting Khmer text read aloud intelligibly is not enough for most production use cases — audiobooks, assistants, dubbing, and similar applications need control over delivery: pacing, pitch, emotional coloring, and non-verbal sounds like laughs or sighs inserted at the right place. This report answers two questions so far: what does VoxCPM2 already give us on Khmer for free (Section 4.1), and can it be taught more (Sections 4.2–4.3).

2.2 Model Selection: Why VoxCPM2

VoxCPM2 is an open-weight text-to-speech model released by OpenBMB under the Apache 2.0 licence, and it is the model this project has adopted for Khmer. Two practical properties motivated that choice.

First, it holds up on Khmer where alternatives fail outright. In the project's four-model comparison, each model was scored for character error rate (CER) on the same fixed set of eight Khmer sentences (Table 1). Fish Audio S2-Pro and Meta MMS both degrade badly on Khmer; VoxCPM2 does not.

Table 1. Khmer character error rate across the four models this project evaluated, on the same fixed 100-sentence set.

Model	Median CER	Mean CER
VoxCPM2	2.47%	4.48%
Higgs TTS 3	8.28%	9.88%
Meta MMS	25.12%	24.84%
Fish Audio S2-Pro	78.01%	77.81%

Lower is better. Full methodology and caveats (including a known scorer bias toward VoxCPM2) are in the project's evaluation report, not repeated here.

Second, it needs no Khmer-specific preprocessing. VoxCPM2 reads raw text directly and infers the language from the script, so Khmer requires no pronunciation lexicon and no word segmenter — components that do not exist for Khmer in a production-ready form, and that ordinarily consume most of the effort in a low-resource TTS project.

One more property matters for everything that follows: VoxCPM2 never trains on the input text as a prediction target — text is purely a conditioning signal. Practically, this means new markup can be added to the input (a parenthetical style description, an inline event tag) without disturbing how the model was originally trained. Both control mechanisms described next rely on this.

VoxCPM2 exposes two ways to influence delivery. A parenthetical description placed before the sentence characterises the whole utterance:

```
model.generate(  
    text="(speaking quickly, a high-pitched voice)"  
    "ថ្ងៃនេះអាកាសធាតុល្អណាស់។")
```

and a bracketed tag placed inline marks a single event at one position in the text:

```
model.generate(text="ខ្ញុំគិតថាមិនអីទេ [laughing] ប៉ុន្តែ...")
```

The documented tag inventory is:

```
[laughing]    [laughter]    [sigh]    [Uhm]    [Shh]  
[Question-ah] [Question-ei] [Question-en] [Question-oh]  
[Surprise-wa] [Surprise-yo] [Dissatisfaction-hnn]
```

OpenBMB's documentation covers both mechanisms only for Chinese and English; it says nothing about Khmer. Their behaviour on Khmer is therefore something this project has to measure rather than assume, which Sections 4.1–4.3 do.

3 Speech Control Framework

3.1 Scope and Definitions

"Speech control" covers several distinct capabilities. This report separates three that matter for this project — prosody, emotion, and non-verbal vocalization — using one distinction that recurs throughout: whether the thing being controlled is a **global attribute** or a **local event**.

A global attribute is a property of the whole utterance (or a long span of one) — pitch register, speaking rate, emotional tone. A local event is bounded: it starts, occupies a short interval, and ends, tied to one specific position in the text — a laugh, a sigh, a filled pause. The distinction is not just about duration: a global attribute has to compete for influence over frames the surrounding speech already mostly determines, whereas a local event is the only thing that determines the frames it occupies. That difference is why the two are harder or easier to train, and it comes back in Section 5.

Three further categories — voice quality, emphasis, and pause timing — are adjacent to the three above but out of scope for this report: VoxCPM2 provides no syntax for them today, and adding one is a larger undertaking than extending an existing tag.

3.2 Prosody

Prosody covers rate, pitch, loudness, pitch variation and pause placement — everything about delivery once the words themselves are set aside.

Table 2. Prosodic dimensions and how this project measures them.

Dimension	What it means	Measurement used here
Speaking rate	how fast the sentence is spoken	Khmer characters per second
Pitch register	how high or low the voice is set	median F0, hertz
Pitch variation	how much the pitch moves	F0 standard deviation, semitones
Loudness	signal level	root-mean-square level, dBFS
Phrasing	pause placement and duration	inter-pausal unit statistics

Prosody is a global attribute, and — as Section 4.1 shows — it is the capability VoxCPM2 already handles on Khmer without any further work.

3.3 Emotion

Emotion is the affective state conveyed by delivery — typically a small closed set such as neutral, happy, angry, sad, surprised. It overlaps with prosody (anger and excitement share elevated pitch and energy) but does not reduce to it: the same rate/pitch/energy setting can read as sarcastic, pleased, or resigned depending on voice quality and timing that a prosody knob does not capture.

In practice, work on emotional TTS depends almost entirely on acted, parallel, utterance-labelled emotional speech corpora, and building one for Khmer is not proportionate to the value it would return right now. VoxCPM2's parenthetical channel would plausibly accept a description like (*sounding*

angry), since it is the same free-text field that already carries (*speaking quickly*), but this has not been tested. Emotion is therefore out of scope for this phase of work — see Section 5.

3.4 Non-Verbal Vocalization

A non-verbal vocalization is a sound a speaker makes that is not a word: laughter, a sigh, an audible breath, a filled pause, a cough, a gasp, a sob, a hesitation particle. These carry stance, regulate turn-taking, and convey affect, and their presence is a big part of what makes conversational speech sound conversational rather than read aloud.

Table 3. Non-verbal vocalization types. Tags in the first five rows are documented in VoxCPM2; the sixth row is not.

Type	Tag	What it communicates
Laughter	[laughing]	amusement, affiliation, mitigation
Sigh	[sigh]	resignation, fatigue, relief
Filled pause	[Uhm]	planning, hesitation, floor-holding
Attention marker	[Shh]	silencing, conspiratorial framing
Discourse particle	[Question-ah], [Surprise-wa], [Dissatisfaction-hnn]	stance, back-channelling, question marking
Breath, gasp, cough, sob	—	phrasing, surprise, distress, physical state

Unlike prosody, non-verbal vocalization is a **local event**: it happens at one place and leaves the speech before and after it unaffected. That makes it an easier training target than a global attribute, because the tagged event has no competing explanation for the frames it occupies — the tag is the only signal available for them. Whether the tags actually fire on Khmer text is untested, and Sections 4.2–4.3 measure whether the underlying mechanism can be taught to work at all, in English first.

4 Experiments

Three experiments were run. The first measures a capability VoxCPM2 already has, on Khmer. The other two test whether a new capability — inline non-verbal vocalization tags — can be taught at all, and whether a first attempt at teaching it generalizes.

4.1 Baseline Prosodic Control on Khmer

The parenthetical mechanism was evaluated on the unmodified model, using eight sentences drawn from the project's fixed Khmer evaluation set. For each of four prosodic axes, three prompts were written to span the axis (Table 4). Every sentence was synthesised under every prompt at three random seeds, giving 96 generations, and the median taken within each cell of the design. Pitch, its variation, loudness, and speaking rate were measured directly from the audio.

Table 4. Prompts used to span each prosodic axis.

Axis	Level 0	Level 1	Level 2
Pitch	<i>(a low-pitched voice)</i>	<i>(a normal-pitched voice)</i>	<i>(a high-pitched voice)</i>
Energy	<i>(speaking softly, quietly)</i>	<i>(speaking at a normal volume)</i>	<i>(speaking loudly)</i>
Rate	<i>(speaking slowly)</i>	<i>(speaking at a normal pace)</i>	<i>(speaking quickly)</i>
Variation	<i>(a flat, monotone delivery)</i>	<i>(a normal delivery)</i>	<i>(a lively, expressive delivery)</i>

Table 5. Response of the unmodified model to a parenthetical prompt on Khmer. The corpus bound is the separation between the low and high bands of this project's hand-labelled Khmer corpus, and represents an upper limit on what fine-tuning against that corpus could teach.

Axis	Unit	Level 0	Level 1	Level 2	Change	Seed s.d.	Corpus bound	ρ	p
Pitch	Hz	147.80	202.81	254.16	+106.37	28.78	28.27	+0.759	0.0002
Energy	dBFS	-20.38	-16.30	-15.89	+4.50	5.69	5.81	+0.605	0.0022
Rate	char/s	14.67	15.84	16.96	+2.29	1.58	6.57	+0.590	0.0035
Variation	st	4.55	4.06	3.92	-0.63	0.68	1.71	-0.273	0.2039

ρ is the Spearman rank correlation between prompt level and measured value; p is a permutation test over 10,000 relabellings. Seed s.d. is the mean run-to-run standard deviation within a cell.

Three findings follow.

Pitch is controlled reliably. The separation between the extreme prompts is 106.37 Hz with a rank correlation of +0.759 ($p = 0.0002$). This exceeds the corpus bound of 28.27 Hz by a factor of approximately four — a fine-tune on this project's own labelled data could not improve on this axis.

Energy and **Speaking Rate** are controlled in aggregate but not per generation. Both show significant rank correlations (+0.605 and +0.590), but the energy effect of 4.50 dB is smaller than the 5.69 dB standard deviation observed across seeds within a single cell. The ordering of the levels is dependable; the value of any individual synthesis is not.

Pitch Variation is not controlled. The correlation is negative (-0.273) and not significant ($p = 0.20$). Prompts requesting a lively delivery produced marginally less pitch movement than prompts requesting a monotone one, consistent with the axis being unaddressed rather than inverted.

Inline tag behaviour on Khmer was not measured here; Sections 4.2–4.3 test the underlying tagging mechanism in English first, as a prerequisite.

4.2 Tag Conditioning: Capability Test

A conventional fine-tune had already been performed on 3.51 hours of tagged English speech derived from deepvk/NonverbalTTS, evaluated on 150 clips drawn from its own training set, and its effect on the inline tag was close to nothing. Deleting the tag from the transcript raised the loss on only 82 of 150 clips — no better than chance ($p = 0.144$) — and moving the tag to the end of the sentence made no difference either ($p = 0.284$).

That null result has two possible explanations, and generated audio alone cannot tell them apart: a dependency that was never learned sounds the same as one that was learned and then washed out at generation time.

- (1) **The mechanism is absent.** The model has no way to bind an event to a specific position in the text. If true, inline tags are not achievable at any corpus size, and the project should fall back to global descriptors of the kind Section 4.1 measures.
- (2) **The signal is too weak.** The mechanism exists, but 98 examples of the most common non-laughter tag, spread over 3.51 hours, is too sparse for the model to pick up on. If true, the approach is sound and the fix is more/better data.

Deliberate overfitting is the cheapest way to tell these apart: memorize a small set of recordings, then check whether the tag's influence shows up on those same recordings. A model that still ignores the tag after seeing the answer hundreds of times has problem (1). A model that picks it up has problem (2), which is fixable.

Table 6 summarises the run. It deliberately does everything a production fine-tune would not: no mix of untagged data (its only job is preventing regression, which this run isn't worried about), no weight decay, a 5x learning rate, and validation on the training clips themselves, so validation loss reads memorization rather than generalization.

Table 6. Configuration of the overfitting run. The full file is `finetune/conf/nvv_overfit.yaml`.

Quantity	Value	Rationale
Clips	60 (5.4 min)	12 per tag; small enough to memorize in about an hour
Tags	5	four undocumented, one documented as control
Untagged mixture	none	regularization is not wanted here
Learning rate	0.0005	five times the documented rate for this adapter
Weight decay	0	no regularization, deliberately
Warmup	50 steps	short, for the same reason
Steps	2,000	about 267 passes over the same clips

Quantity	Value	Rationale
Fine-tune scope	rank 64 (alpha 64)	a lightweight adapter covering the relevant model components
Validation set	identical to training	validation loss reads memorization by design

Clips were admitted only if they carried exactly one tag, occurring exactly once, in a 2–8 second recording — the cleanest possible mapping from one string to one event. Four of the five tags used — [cough], [groan], [throat-clear], [sniff] — appear nowhere in the inventory OpenBMB publishes (Table 3). The fifth, [laughing], is documented and kept as a control: the unmodified model already produces laughter, so if it trained appreciably better than the undocumented tags, that would suggest the model was retrieving prior knowledge of the word rather than learning a new binding.

This test was run in English rather than Khmer, on purpose: it asks only whether the tagging mechanism itself can be taught to work at all, not whether it works specifically in Khmer. That second question is deferred until this one is settled.

The instrument is a forward pass, one per clip per condition, comparing the loss across four rewritings of the transcript (Table 7). Generated audio is not used to decide the question, for the reason given above.

Table 7. The four conditions of the sensitivity probe.

Condition	Transcript	What it tests
True	tag inline, at the event	reference
Removed	tag deleted	whether the tag carries information at all
Moved	tag pushed to the end of the sentence	whether the tag's position carries information
Scrambled	word order destroyed, tags left in place	positive control

The **moved** condition is what distinguishes a local event from an utterance-level flag: a model that only learned "this clip contains a cough" scores the same when the tag is relocated, whereas a model that learned *where* the cough belongs does not. The **scrambled** condition is the sanity check: it must raise the loss, or the probe isn't measuring anything and no other row can be trusted.

Results

Table 8. Effect of each rewriting on the loss after memorization (n = 60; reference loss 0.23070). *Worse* counts the clips on which the rewriting raised the loss; *p* is a two-sided sign test.

Condition	Mean loss	Change	Worse	p
Removed	0.34955	+0.11885	59/60	< 0.0001
Moved	0.29089	+0.06019	43/60	0.0005
Scrambled	0.84472	+0.61402	59/60	< 0.0001

The positive control raised the loss by +0.61402 on 59 of 60 clips, so the probe is measuring the objective and the other two rows can be read.

The tag now carries information. Deleting it raises the loss on 59 of 60 clips. **Its position matters too** — the more important finding: moving the tag without deleting it raises the loss on 43 of 60 clips, at 51% of the cost of deleting it — whereas the same test on the unmodified model gives $p = 0.98$. The model isn't just flagging "this utterance contains a cough"; it's reading where in the sentence the tag sits and placing the event there.

Absolute losses aren't comparable across model states, evaluated on different clips at different points in training. Table 9 instead compares the cost of deleting the tag as a fraction of the cost of destroying the whole transcript — a ratio internal to each model.

Table 9. Saliency of the inline tag relative to the transcript, across the three states of the model. The ratio is the cost of deleting the tag divided by the cost of scrambling the transcript, measured within each model. All three rows are measured on clips the model in question was trained on; the held-out comparison is reported separately in Section 4.3.

Model state	n	Deletion p	Relocation p	Tag / transcript
Unmodified	150	0.2313	0.9796	2.3%
Fine-tuned, 3.51 h	150	0.1442	0.2839	5.8%
Overfitted, 5.4 min	60	< 0.0001	0.0005	19.4%

The two p-value columns are sign tests on the deletion and relocation conditions respectively. The overfitted figure is 3.3 times the fine-tuned one and 8.3 times the unmodified one.

Table 10. Loss under the deletion condition, by tag. The control tag is the only one OpenBMB documents.

Tag	Documented	Mean loss when deleted
[throat-clear]	no	0.3745
[sniff]	no	0.3619
[cough]	no	0.3618
[laughing]	yes	0.3269
[groan]	no	0.3226

The four undocumented tags are not weaker than the documented one, which argues that the effect in Table 8 is a newly learned binding rather than the model retrieving prior knowledge of the English words in brackets.

Interpretation and limits

This experiment shows VoxCPM2 *can* bind an arbitrary bracketed tag to a specific non-verbal event at a specific position in a sentence, and that a lightweight fine-tune is enough to install that binding. The "mechanism is absent" explanation from above is ruled out: the earlier null result was a problem with the training data, not the model. That's the finding the project needed, obtained for the cost of an hour of training.

It proves nothing about generalization, and should not be quoted as though it did. The evaluation runs on the same 60 sentences the model was trained on, roughly 267 times each — that's what overfitting means. Whether the same tags work on text the model hasn't seen is a separate question, addressed next.

4.3 Tag Conditioning: Generalization Test

The generalization test Section 4.2 deferred to has since been run, and the result is negative. The validation split of the corpus was never shown to the model during training; 28 of its clips carry an inline tag, and those 28 are the entire available sample. The same four-condition probe was applied to the fine-tuned model and to the unmodified model on those identical clips, so the two are compared within one clip set rather than across sets. Table 11 reports the outcome.

Table 11. Tag sensitivity on 28 clips held out of training, for the fine-tuned model and the unmodified model over the same clips. The two are indistinguishable.

Model state	Reference loss	Deletion Δ	Clips worse	p	Tag / transcript
Unmodified	0.87491	+0.01603	19/28	0.0436	51.9%
Fine-tuned	0.85221	+0.01639	18/28	0.0925	61.9%

The two rows describe the same behaviour. The cost of deleting the tag differs between them by only 0.0004, and the sign-test counts differ by a single clip in the unmodified model's favour. Whatever sensitivity to the tag these sentences show, the model had it before training — 2,000 steps on 3.51 hours added nothing that reaches text the model hasn't seen.

Two caveats. First, statistical power: at $n = 28$, a sign test needs roughly twenty clips of twenty-eight to reach significance, so a small true effect could be hiding here — what this rules out is a large one, which is what a working method should produce given the capability shown in Section 4.2. Second, the final column isn't comparable to the same column in Table 9: these are different sentences, and the unmodified model scores 52% here against 2% on the clips used there — which is why the unmodified model was re-measured on this specific held-out set rather than reusing the earlier number.

Together, Sections 4.2 and 4.3 tell a consistent story: the mechanism exists — VoxCPM2 can bind a bracketed tag to a local event and to its position — but the fine-tune as configured does not install that binding in a form that survives to unseen text. The fixes below are prerequisites, not refinements.

Provenance. The figures in Sections 4.2–4.3 are read at build time from `finetune/results/nvv/tag_sensitivity_overfit.json` and its counterparts for the other model states and the held-out set. The trained adapter itself was deleted in error after the run and has not been regenerated; the recorded measurements and the synthesized audio survive, but reproducing the audio would require repeating the training. The procedure is documented in `finetune/OVERFIT_EXPERIMENT.md`.

5 Findings

Of the three speech-control categories in Section 3, one is already handled by the base model, one is blocked on data, and one has a working mechanism whose training recipe does not yet generalize. This section restates what each experiment established; it will be extended as further experiments are run.

Prosody is already handled. The parenthetical mechanism moves Khmer pitch by 106.37 Hz at a rank correlation of +0.759, roughly four times the 28.27 Hz this project's own labelled corpus could express (Section 4.1). A prosody fine-tune trained on that corpus would reproduce a capability the base model already has, in a conditioning channel that's already in use. What remains — pitch variation and seed-to-seed consistency — is real but small.

Emotion is blocked on data, not method. Every reliable result in this space depends on acted, parallel, labelled emotional speech, and no Khmer corpus of that kind exists (Section 3.3).

Non-verbal vocalization has a mechanism that works but a recipe that doesn't yet generalize. It is a local event, not a global attribute (Section 3.1) — the tagged event has no competing explanation for the frames it occupies, unlike prosody or emotion. The interface already exists: `[laughing]`, `[sigh]` and `[Uhm]` are documented in the unmodified model. Section 4.2 shows the underlying binding can be taught; Section 4.3 shows the specific recipe tried does not carry that binding to text the model hasn't seen. Whether a different recipe closes that gap is open, and is the next thing this document will report on.

References

- Gillick, J., Deng, W., Ryokai, K. and Bamman, D. (2021). Robust laughter detection in noisy environments. *Interspeech 2021*, 2481–2485.
- Gong, Y., Yu, J. and Glass, J. (2022). VocalSound: a dataset for improving human vocal sounds recognition. *ICASSP 2022*. arXiv:2205.03433.
- Guo, Z., Leng, Y., Wu, Y., Zhao, S. and Tan, X. (2023). PromptTTS: controllable text-to-speech with text descriptions. *ICASSP 2023*. arXiv:2211.12171.
- Hsu, C.-C., Kanda, N., Zhu, Y. et al. (2024). Laugh Now Cry Later: controlling time-varying emotional states of flow-matching-based zero-shot text-to-speech. *IEEE SLT 2024*. arXiv:2407.12229.
- Ji, S., Zuo, J., Fang, M. et al. (2024). TextrolSpeech: a text style control speech corpus with codec language text-to-speech models. *ICASSP 2024*. arXiv:2308.14430.
- Kanda, N., Wang, X., Eskimez, S. E. et al. (2024). Making flow-matching-based zero-shot text-to-speech laugh as you like. arXiv:2402.07383.
- Leng, Y., Guo, Z., Shen, K. et al. (2024). PromptTTS 2: describing and generating voices with text prompt. *ICLR 2024*. arXiv:2309.02285.
- Lyth, D. and King, S. (2024). Natural language guidance of high-fidelity text-to-speech with synthetic annotations. arXiv:2402.01912. Released as Parler-TTS.
- MNV-17: a high-quality performative Mandarin dataset for nonverbal vocalization recognition in speech (2025). arXiv:2509.18196.
- NonverbalTTS: a public English corpus of text-aligned nonverbal vocalizations with emotion annotations for text-to-speech (2025). *Speech Synthesis Workshop 2025*. arXiv:2507.13155.
- NVMOS: non-verbal vocalization quality assessment in speech (2026). arXiv:2606.15888.
- NVSpeech: an integrated and scalable pipeline for human-like speech modeling with paralinguistic vocalizations (2025). arXiv:2508.04195.
- NVV-SuperBench: beyond words, beyond quality — benchmarking nonverbal vocalizations in speech generation (2026). arXiv:2604.16211.
- OpenBMB (2025). VoxCPM2 model card. huggingface.co/openbmb/VoxCPM2.
- Yang, D., Liu, S., Huang, R. et al. (2023). InstructTTS: modelling expressive TTS in discrete latent space with natural language style prompt. arXiv:2301.13662.
- Zhou, K., Sisman, B., Liu, R. and Li, H. (2022). Emotional voice conversion: theory, databases and ESD. *Speech Communication*, 137, 1–18. arXiv:2105.14762.